LAB2 Частина 4. Оцінка параметрів функції

Виконавець: **Тижук Дмитро Іванович**, група **К-26**
Викладач: **Ляшко А. В.**

Знадобляться бібліотеки `numpy`, `scipy`, `pandas`, `matplolib`.

In [1]:
```
from pathlib import Path
import numpy as np
import scipy, scipy.linalg as la
import pandas as pd
import matplotlib, matplotlib.pyplot as plt
print(f'numpy {np.__version__}, pandas {pd.__version__}, scipy {scipy.__v
```

```
numpy 2.4.2, pandas 3.0.1, scipy 1.17.1, matplotlib 3.10.8
```

Постановка задачі

У результаті кількох експериментів з обчислення швидкодії програми була отримана низка даних вигляду (n, t), де
n — розмір оброблюваних вхідних даних,
t — кількість часу, що була в середньому необхідна для їх обробки.

Дослідник знає, що часова складність програми повинна мати вигляд $T(n) = \Theta(n^a)$. Знайти значення параметра a.

Зауваження. Якщо $T(n) = \Theta(n^a)$, то можна вважати, що $T(n) \approx cn^a$.

На відміну від частини 1, у цій задачі результати експериментів задані у csv-файлі. Кожен рядок файлу містить розмір оброблюваних даних та час роботи на них. Для деяких розмірів виконували кілька експериментів, тому розміри можуть повторюватись. Під час оцінювання параметрів для кожного наявного розміру слід використовувати середнє значення часу роботи.

In [2]:
```
path = Path("./lab2_var092.csv")
```

Завантажити дані в DataFrame за допомогою функції `pd.read_csv` (документація тут).

In [3]:
```
raw = pd.read_csv(path, encoding="utf-8", header=0, sep=",")
raw
```

Out[3]:

	n	t
0	825	1633
1	155	351
2	10	28
3	455	978
4	74	195
...	...	...
73	18	63
74	365	755
75	548	1309
76	10	37
77	548	1287

78 rows × 2 columns

Виконати необхідні перетворення даних засобами `pandas`

In [4]:
```
processed_data = raw.groupby(by="n").mean()
processed_data.reset_index(inplace=True)
processed_data
```

Out[4]:

	n	t
0	10	33.666667
1	18	58.666667
2	28	87.000000
3	67	195.500000
4	70	200.000000
5	74	180.000000
6	79	211.000000
7	119	268.000000
8	155	379.333333
9	213	472.000000
10	236	538.000000
11	238	513.500000
12	288	688.750000
13	341	659.000000
14	365	743.000000
15	373	642.000000
16	377	891.000000
17	382	841.666667
18	384	832.000000
19	455	970.500000
20	494	1013.666667
21	548	1190.250000
22	591	1223.750000
23	661	1351.666667
24	667	1103.000000
25	732	1612.500000
26	776	1451.000000
27	818	1413.000000
28	825	1545.500000
29	898	1779.000000
30	918	2028.000000
31	1000	2126.500000

In [5]:

```
processed_data["ones"] = 1
processed_data
```

Out[5]:

	n	t	ones
0	10	33.666667	1
1	18	58.666667	1
2	28	87.000000	1
3	67	195.500000	1
4	70	200.000000	1
5	74	180.000000	1
6	79	211.000000	1
7	119	268.000000	1
8	155	379.333333	1
9	213	472.000000	1
10	236	538.000000	1
11	238	513.500000	1
12	288	688.750000	1
13	341	659.000000	1
14	365	743.000000	1
15	373	642.000000	1
16	377	891.000000	1
17	382	841.666667	1
18	384	832.000000	1
19	455	970.500000	1
20	494	1013.666667	1
21	548	1190.250000	1
22	591	1223.750000	1
23	661	1351.666667	1
24	667	1103.000000	1
25	732	1612.500000	1
26	776	1451.000000	1
27	818	1413.000000	1
28	825	1545.500000	1
29	898	1779.000000	1
30	918	2028.000000	1
31	1000	2126.500000	1

In [6]:
```
n_min = processed_data["n"].min()
```

In [7]:
```
n_max = processed_data["n"].max()
```

In [8]:
```
n_min, n_max
```

Out[8]:
```
(np.int64(10), np.int64(1000))
```

Побудувати матрицю $data$ (див. LSM.pdf). Має бути багатовимірним масивом типу `ndarray`.

▶ Підказка.

In [9]:
```
data = processed_data[["ones", "n", "t"]].to_numpy()
data[:5]
```

Out[9]:
```
array([[  1.        ,  10.        ,  33.66666667],
       [  1.        ,  18.        ,  58.66666667],
       [  1.        ,  28.        ,  87.        ],
       [  1.        ,  67.        , 195.5       ],
       [  1.        ,  70.        , 200.        ]])
```

In [10]:
```
x = np.log(data[:, 1])
y = np.log(data[:, 2])
```

In [11]:
```
X = np.concatenate((data[:, 0].reshape(-1, 1), x.reshape(-1, 1)), axis=1)
Y = y.reshape((-1, 1))
```

In [12]:
```
A = X.T @ X
B = X.T @ Y
A, B
```

Out[12]:
```
(array([[  32.        ,  178.12620212],
        [ 178.12620212, 1036.07181754]]),
 array([[ 203.96950238],
        [1174.17020578]]))
```

In [13]:
```
C = la.solve(A, B)
C
```

Out[13]:
```
array([[1.52704046],
       [0.87075459]])
```

Знайти оцінки параметрів a, c функції $T(n) \approx cn^a$.

In [14]:
```
a = C[1, 0]
c = np.exp(C[0, 0])
```

In [15]:
```
print(f'{c = :.6f}, {a = :.6f}')
```

```
c = 4.604529, a = 0.870755
```

Візуальна оцінка якості наближення

Для цього побудуємо графік знайденої функції та накладемо на нього вхідні дані.

Побудувати функцію `T_estimated`, що обчислює час роботи програми за формулою $T(n) = cn^a$ з використанням знайдених оцінок `a`, `c` параметрів a та c.

In [16]:
```
def T_estimated(n):
    return c * np.power(n, a)
```

Для візуалізації будемо будувати графік поточково, тому потрібні значення в проміжних точках.

Почнемо з того, що визначимо точки, які використаємо в поточковій побудові графіку. Отриманий масив точок, які рівномірно розташовані по всій шкалі, запам'ятаємо в змінній `scale`.

In [17]:
```
scale = np.linspace(n_min, n_max, 500)
```

З використанням функції `T_estimated` обчислимо розрахункові значення функції часової складності на всій шкалі і запам'ятаємо в змінній `times`.

▶ Підказка.

In [18]:
```
times = T_estimated(scale)
```

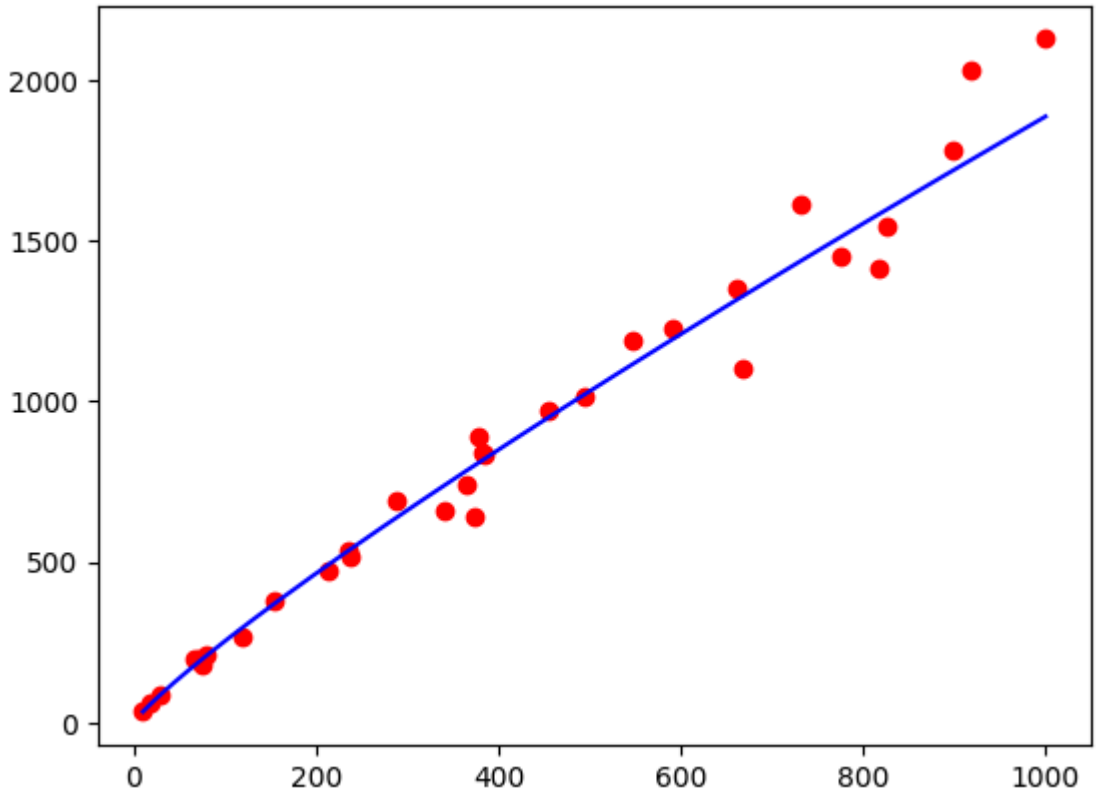
Візуалізуємо отримані результати.

На графіку червоні точки відповідають "експериментальним" даним. Синім зображено графік отриманої залежності часу роботи від розмірів даних.

n — масив вхідних розмірів
t — масив, що містить значення часу роботи на даних відповідного розміру (зміст цих змінних такий самий, що і в частині 1)

In [19]:
```
n = data[:, 1]
t = data[:, 2]
plt.plot(n, t, color="red", marker='o', linestyle='None')
plt.plot(scale, times, color='blue')
#plt.show()
#plt.savefig('1.png')
```

Out[19]:
```
[<matplotlib.lines.Line2D at 0x1242b0590>]
```



Частину 4 зроблено!